

MODULE 2

ERROR DETECTION AND CORRECTION

Data can be corrupted during transmission. Some applications require that errors be detected and corrected.

Types of Errors

Whenever bits flow from one point to another, they are subject to unpredictable changes because of interference. This interference can change the shape of the signal. Errors are of two types:

Single-Bit Error

The term single-bit error means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1.

For a single-bit error to occur, the noise must have a duration of only 1) ls, which is very rare; noise normally lasts much longer than this.

Burst Error

The term burst error means that 2 or more bits in the data unit have changed from 1 to 0 or from 0 to 1.

A burst error is more likely to occur than a single-bit error. The duration of noise is normally longer than the duration of 1 bit, which means that when noise affects data, it affects a set of bits. The number of bits affected depends on the data rate and duration of noise.

Redundancy

The central concept in detecting or correcting errors is redundancy.

To be able to detect or correct errors, we need to send some extra bits with our data.

These redundant bits are added by the sender and removed by the receiver. Their presence allows the receiver to detect or correct corrupted bits.

Detection versus Correction

The correction of errors is more difficult than the detection. In error **detection**, we are looking only to see if any error has occurred. The answer is a simple yes or no.

In error **correction**, we need to know the exact number of bits that are corrupted and more importantly, their location in the message. The number of the errors and the size of the message are important factors.

Forward Error Correction versus Retransmission

There are two main methods of error correction.

Forward error correction is the process in which the receiver tries to guess the message by using redundant bits. This is possible, as we see later, if the number of errors is small.

Correction by **retransmission** is a technique in which the receiver detects the occurrence of an error and asks the sender to resend the message. Resending is repeated until a message arrives that the receiver believes is error-free (usually, not all errors can be detected).

Coding

Redundancy is achieved through various coding schemes. The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits. The receiver checks the relationships between the two sets of bits to detect or correct the errors.

In modular arithmetic, we use only a limited range of integers. We define an upper limit, called a modulus N . We then use only the integers 0 to $N - 1$, inclusive. This is modulo- N arithmetic.

For example, if the modulus is 12, we use only the integers 0 to 11, inclusive.

Of particular interest is modulo-2 arithmetic. In this arithmetic, the modulus N is 2. We can use only 0 and 1.

Addition	Subtraction
----------	-------------

$0+0=0$	$0-0=0$
---------	---------

$1+0=1$	$1-0=1$
---------	---------

$1+1=0$	$1-1=0$
---------	---------

$$0+1=1$$

$$0-1=1$$

Particularly that addition and subtraction give the same results. In this arithmetic we use the XOR (exclusive OR) operation for both addition and subtraction. The result of an XOR operation is 0 if two bits are the same; the result is 1 if two bits are different.

If the modulus is not 2, addition and subtraction are distinct.

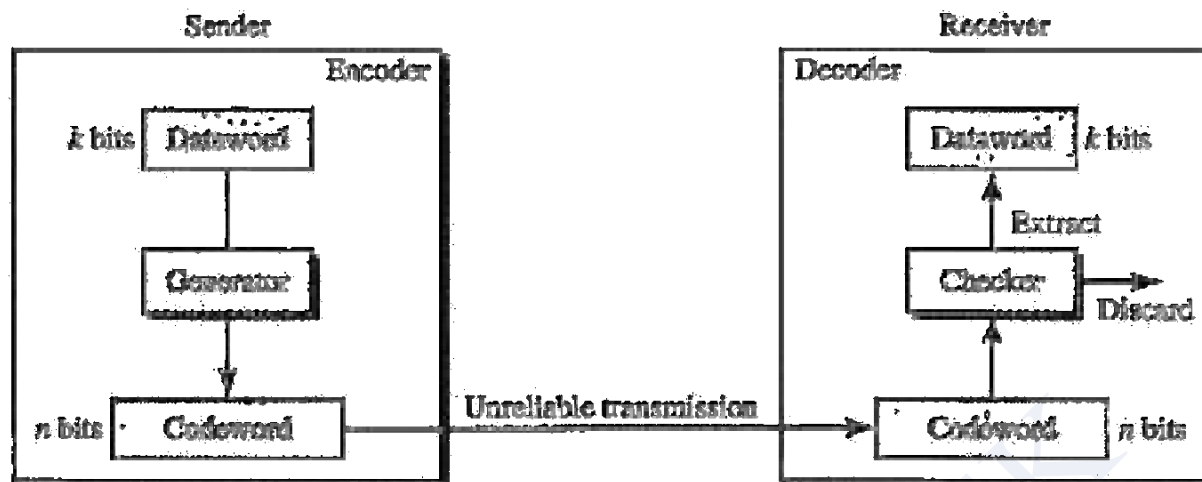
BLOCK CODING

In block coding, we divide our message into blocks, each of k bits, called data words. We add r redundant bits to each block to make the length $n = k + r$. The resulting n -bit blocks are called code words.

With k bits, we can create a combination of 2^k data words; with n bits, we can create a combination of 2^n code words. Since $n > k$, the number of possible code words is larger than the number of possible data words. The block coding process is one-to-one; the same data word is always encoded as the same codeword. This means that we have $2^n - 2^k$ code words that are not used.

Error Detection

The sender creates code words out of data words by using a generator that applies the rules and procedures of encoding. Each codeword sent to the receiver may change during transmission. If the received codeword is the same as one of the valid code words, the word is accepted; the corresponding data word is extracted for use. If the received codeword is not valid, it is discarded. However, if the codeword is corrupted during transmission but the received word still matches a valid codeword, the error remains undetected. This type of coding can detect only single errors. Two or more errors may remain undetected.



[Diagram: Structure of encoder and decoder for Error detection]

Example

Let us assume that $k = 2$ and $n = 3$. Table 1 shows the list of data words and code words. Later, we will see how to derive a codeword from a data word.

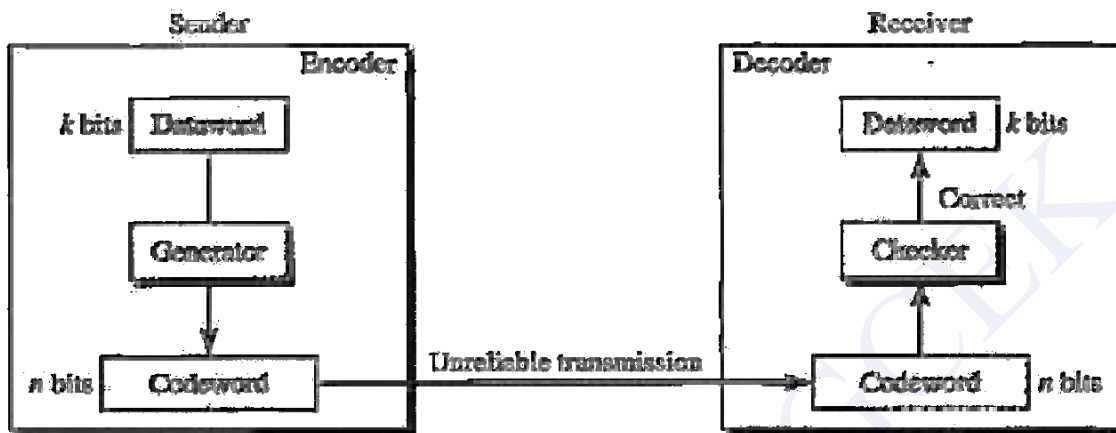
Data words	Code words
00	000
01	011
10	101
11	110

Assume the sender encodes the data word 01 as 011 and sends it to the receiver. Consider the following cases:

1. The receiver receives 011. It is a valid codeword. The receiver extracts the data word 01 from it.
2. The codeword is corrupted during transmission, and 111 is received (the leftmost bit is corrupted). This is not a valid codeword and is discarded.
3. The codeword is corrupted during transmission, and 000 is received (the right two bits are corrupted). This is a valid codeword. The receiver incorrectly extracts the data word 00. Two corrupted bits have made the error undetectable.

Error Correction

As we said before, error correction is much more difficult than error detection. In error detection, the receiver needs to know only that the received codeword is invalid; in error correction the receiver needs to find (or guess) the original codeword sent.



[Diagram: Structure of encoder and decoder for Error correction]

Example

Let us assume that $k=2$ and $n=3$. Below table shows the list of data words and code words.

Dataword(k)	Codeword($n=k+r$)
00	00000
01	01011
10	10101
11	11110

Assume the dataword is 01. The sender consults the table (or uses an algorithm) to create the codeword 01011. The codeword is corrupted during transmission, and 01001 is received (error in the second bit from the right). First, the receiver finds that the received codeword is not in the table. This means an error has occurred. (Detection must come before correction.) The receiver, assuming that there is only 1 bit corrupted, uses the following strategy to guess the correct dataword.

1. Comparing the received codeword with the first codeword in the table (01001 versus 00000), the receiver decides that the first codeword is not the one that was sent because there are two different bits.

2. By the same reasoning, the original codeword cannot be the third or fourth one in the table.
3. The original codeword must be the second one in the table because this is the only one that differs from the received codeword by 1 bit. The receiver replaces 01001 with 01011 and consults the table to find the dataword 01.

Hamming Distance

The Hamming distance between two words (of the same size) is the number of differences between the corresponding bits.

The Hamming distance can easily be found if we apply the *XOR operation on the two words and count the number of 1s in the result*. Note that the Hamming distance is a value greater than zero.

The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.

Example:

Let us find the Hamming distance between two pairs of words.

1. The Hamming distance $d(000, 011)$ is 2 because $000 \text{ XOR } 011$ is 011 (two 1s).
2. The Hamming distance $d(10101, 11110)$ is 3 because $10101 \text{ XOR } 11110$ is 01011 (three 1s).

Linear Block Codes

In a linear block code, the exclusive OR (XOR) of any two valid code words creates another valid codeword.

The scheme in Table 1 is a linear block code because the result of XORing any codeword with any other codeword is a valid codeword. For example, the XORing of the second and third code words creates the fourth one.

Let us now show some linear block codes. These codes are trivial because we can easily find the encoding and decoding algorithms and check their performances.

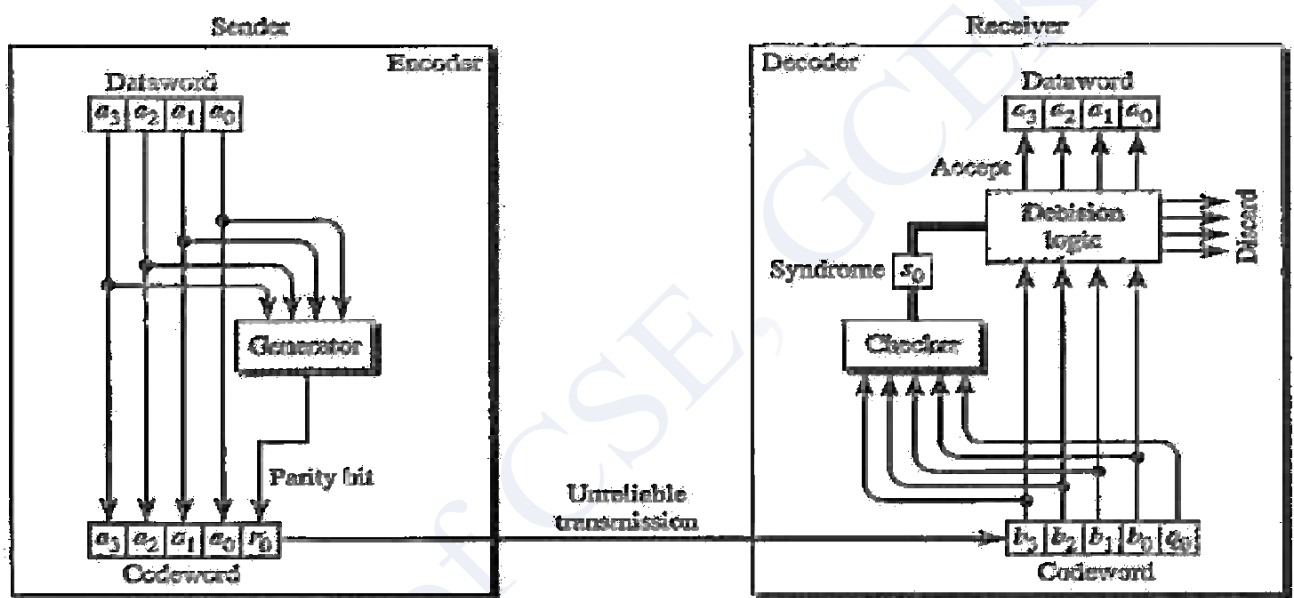
Simple Parity-Check Code

A simple parity-check code is a single-bit error-detecting code in which $n = k + 1$ with $d_{\min} = 2$.

In this code, a k -bit data word is changed to an n -bit codeword where $n = k + 1$. The extra bit, called the parity bit, is selected to make the total number of 1s in the codeword even.

The minimum Hamming distance for this category is $d_{\min} = 2$, which means that the code is a single-bit error-detecting code; it cannot correct any error.

Table 1 is a parity-check code with $k = 2$ and $n = 3$.



[Encoder and Decoder for Simple parity check]

The encoder uses a generator that takes a copy of a 4-bit dataword (a_0, a_1, a_2 and a_3) and generates a parity bit r_0 . The dataword bits and the parity bit create the 5-bit codeword. *The parity bit that is added makes the number of 1s in the codeword even.*

This is normally done by adding the 4 bits of the dataword (modulo-2); the result is the parity bit. In other words,

$$r_0 = a_3 + a_2 + a_1 + a_0 \text{ (modulo 2)}$$

If the number of 1s is even, the result is 0; if the number of 1s is odd, the result is 1. In both cases, the total number of 1s in the codeword is even.

The sender sends the codeword which may be corrupted during transmission. The receiver receives a 5-bit word. The checker at the receiver does the same thing as the generator in the

sender with one exception: The addition is done over all 5 bits. The result, which is called the syndrome, is just 1 bit. *The syndrome is 0 when the number of 1s in the received codeword is even; otherwise, it is 1.*

$$s_0 = b_3 + b_2 + b_1 + b_0 \text{ (modulo 2)}$$

The syndrome is passed to the decision logic analyzer. *If the syndrome is 0*, there is no error in the received codeword; the data portion of the received codeword is accepted as the dataword; *if the syndrome is 1*, the data portion of the received codeword is discarded. The dataword is not created.

Example:

Let us look at some transmission scenarios. Assume the sender sends the dataword 1011. The codeword created from this dataword is 10111, which is sent to the receiver.

We examine five cases:

1. No error occurs; the received codeword is 10111. The syndrome is 0. The dataword 1011 is created.
2. One single-bit error changes a_1 . The received codeword is 10011. The syndrome is 1. No dataword is created.
3. One single-bit error changes r_0 . The received codeword is 10110. The syndrome is 1. No dataword is created.
4. An error changes r_0 and a second error changes a_3 . The received codeword is 00110. The syndrome is 0. The dataword 0011 is created at the receiver. Note that here the dataword is wrongly created due to the syndrome value. *The simple parity-check decoder cannot detect an even number of errors. The errors cancel each other out and give the syndrome a value of 0.*
5. Three bits- a_3 , a_2 , and a_1 -are changed by errors. The received codeword is 01011. The syndrome is 1. The dataword is not created. *This shows that the simple parity check, guaranteed to detect one single error, can also find any odd number of errors.*

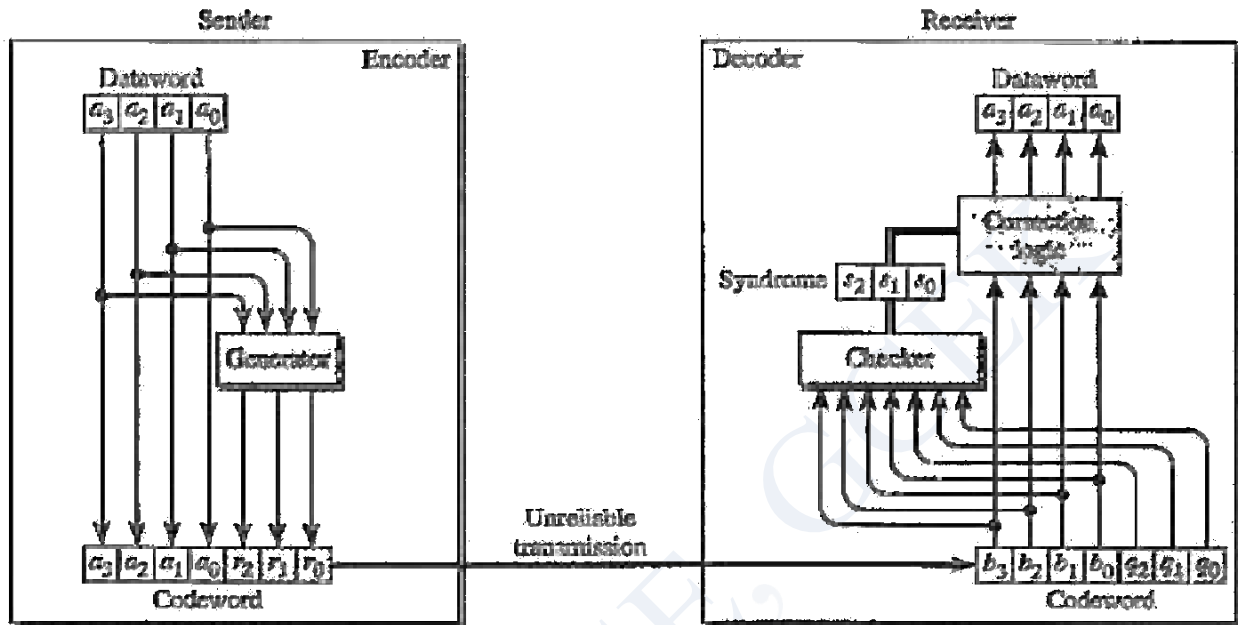
Hamming Code

These codes were originally designed with $d_{\min} = 3$, which means that they can detect up to two errors or correct one single error. Although there are some Hamming codes that can correct more than one error, our discussion focuses on the single-bit error-correcting code.

Let us find the relationship between n and k in a Hamming code. We need to choose an integer $m \geq 3$.

The values of n and k are then calculated from m as $n = 2^m - 1$ and $k = n - m$. The number of check bits $r = m$.

A Hamming code can only correct a single error or detect a double error.



[Encoder and Decoder for Hamming Code]

A copy of a 4-bit dataword is fed into the generator that creates three parity checks r_0 , r_1 and r_2 as shown below:

$$\begin{aligned} r_0 &= a_2 + a_1 + a_0 && \text{modulo-2} \\ r_1 &= a_3 + a_2 + a_1 && \text{modulo-2} \\ r_2 &= a_1 + a_0 + a_3 && \text{modulo-2} \end{aligned}$$

In other words, each of the parity-check bits handles 3 out of the 4 bits of the dataword. The total number of 1s in each 4-bit combination (3 dataword bits and 1 parity bit) must be even.

The checker in the **decoder** creates a 3-bit syndrome ($s_2s_1s_0$) in which each bit is the parity check for 4 out of the 7 bits in the received codeword:

$$\begin{aligned} s_0 &= b_2 + b_1 + b_0 + q_0 && \text{modulo-2} \\ s_1 &= b_3 + b_2 + b_1 + q_1 && \text{modulo-2} \\ s_2 &= b_1 + b_0 + b_3 + q_2 && \text{modulo-2} \end{aligned}$$

The equations used by the checker are the same as those used by the generator with the parity-check bits added to the right-hand side of the equation. The 3-bit syndrome creates eight different bit patterns

(000 to 111) that can represent eight different conditions. These conditions define a lack of error or an error in 1 of the 7 bits of the received codeword, as shown in Table:

Syndrome	000	001	010	011	100	101	110	111
Error	None	q_0	q_1	b_2	q_2	b_0	b_3	b_1

Let us trace the path of three datawords from the sender to the destination:

1. The dataword 0100 becomes the codeword 0100011. The codeword 01 00011 is received. The syndrome is 000 (no error), the final dataword is 0100.
2. The dataword 0111 becomes the codeword 0111001. The codeword 0011001 is received. The syndrome is 011. According to Table, b_2 is in error. After flipping b_2 (changing the 1 to 0), the final dataword is 0111.
3. The dataword 1101 becomes the codeword 1101000. The codeword 0001000 is received (two errors). The syndrome is 101, which means that b_0 is in error. After flipping b_0 , we get 0000, the wrong dataword. This shows that our code cannot correct two errors.

Cyclic Codes

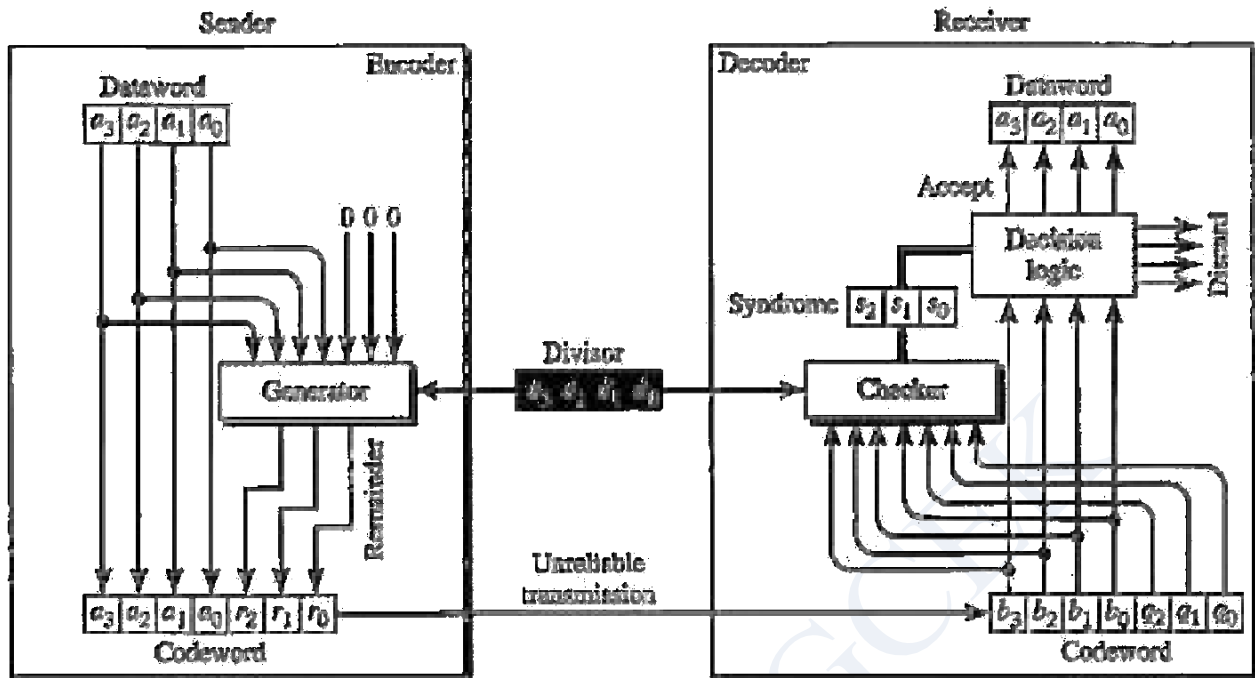
Cyclic codes are special linear block codes with one extra property. In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword. For example, if 1011000 is a codeword and we cyclically left-shift, then 0110001 is also a codeword.

If we call the bits in the first word a_0 to a_6 and the bits in the second word b_0 to b_6 , we can shift the bits by using the following:

$$b_1=a_0 \quad b_2=a_1 \quad b_3=a_2 \quad b_4=a_3 \quad b_5=a_4 \quad b_6=a_5 \quad b_0=a_6$$

Cyclic Redundancy Check

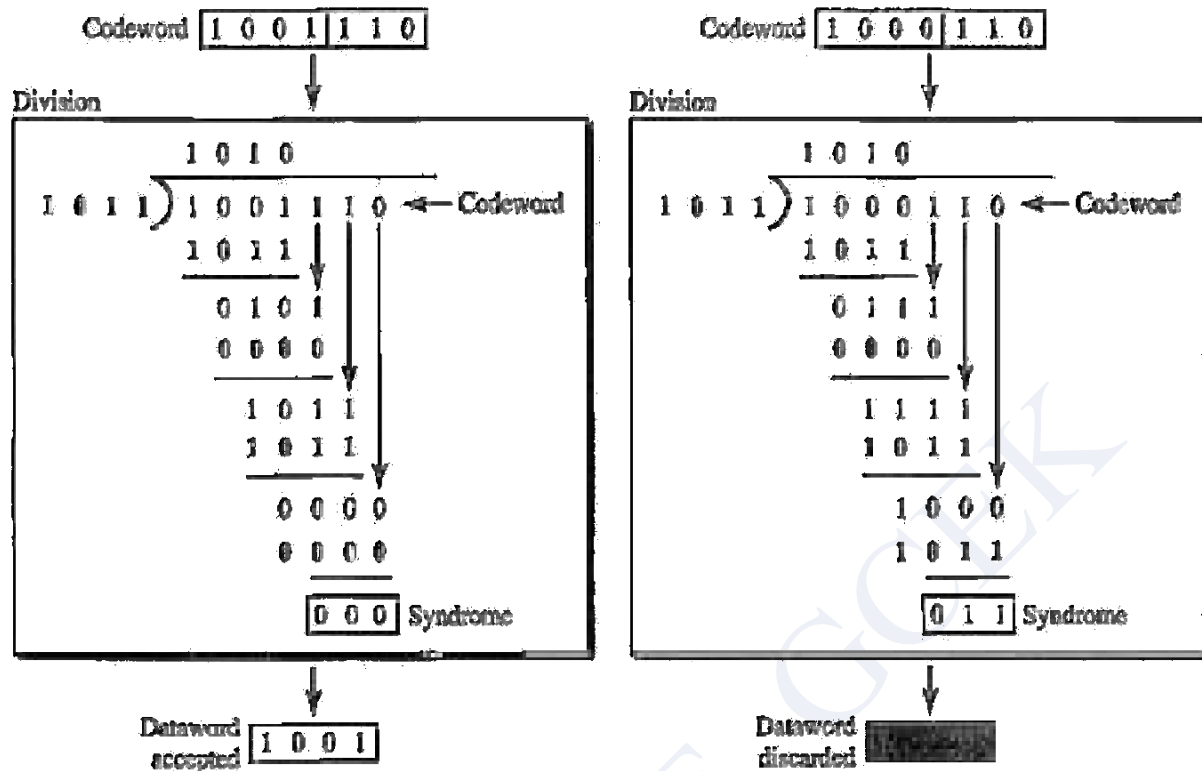
We can create cyclic codes to correct errors. However, the theoretical background required is beyond the scope of this book. In this section, we simply discuss a category of cyclic codes called the cyclic redundancy check (CRC) that is used in networks such as LANs and WANs.



[Encoder and Decoder for CRC]

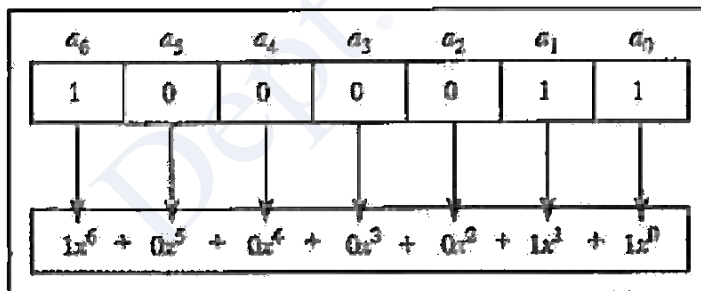
In the encoder, the dataword has k bits (4 here); the codeword has n bits (7 here). The size of the dataword is augmented by adding $n - k$ (3 here) 0s to the right-hand side of the word. The n -bit result is fed into the generator. The generator uses a divisor of size $n - k + 1$ (4 here), predefined and agreed upon. The generator divides the augmented dataword by the divisor (modulo-2 division). The quotient of the division is discarded; the remainder ($r_2r_1r_0$) is appended to the dataword to create the codeword.

The decoder receives the possibly corrupted codeword. A copy of all n bits is fed to the checker which is a replica of the generator. The remainder produced by the checker is a syndrome of $n - k$ (3 here) bits, which is fed to the decision logic analyzer. The analyzer has a simple function. If the syndrome bits are all as, the 4 leftmost bits of the codeword are accepted as the dataword (interpreted as no error); otherwise, the 4 bits are discarded (error).

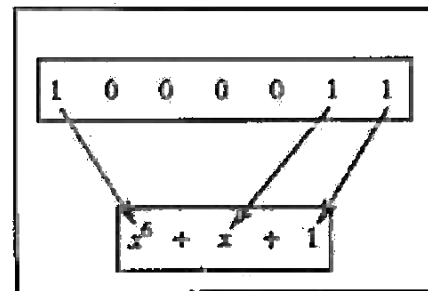


Polynomial

A pattern of 0s and 1s can be represented as a **polynomial** with coefficients of 0 and 1. The power of each term shows the position of the bit; the coefficient shows the value of the bit. Below figure shows a binary pattern and its polynomial representation.



a. Binary pattern and polynomial



b. Short form

DATA LINK CONTROL

The two main functions of the data link layer are **data link control** and **media access control**. The first, data link control, deals with the design and procedures for communication between two adjacent nodes: node-to-node communication.

The second function of the data link layer is media access control, or how to share the link.

Data link control functions include **framing**, **flow** and error **control**, and software implemented protocols that provide smooth and reliable transmission of frames between nodes.

Framing

The data link layer, on the other hand, needs to pack bits into frames, so that each frame is distinguishable from another.

Framing in the data link layer separates a message from one source to a destination, or from other messages to other destinations, by adding a sender address and a destination address. The destination address defines where the packet is to go; the sender address helps the recipient acknowledge the receipt. Frames can be of fixed or variable size.

1. Fixed-Size Framing

In fixed-size framing, there is no need for defining the boundaries of the frames; the size itself can be used as a delimiter.

2. Variable-Size Framing

In variable-size framing, we need a way to define the end of the frame and the beginning of the next. Historically, two approaches were used for this purpose: a character-oriented approach and a bit-oriented approach.

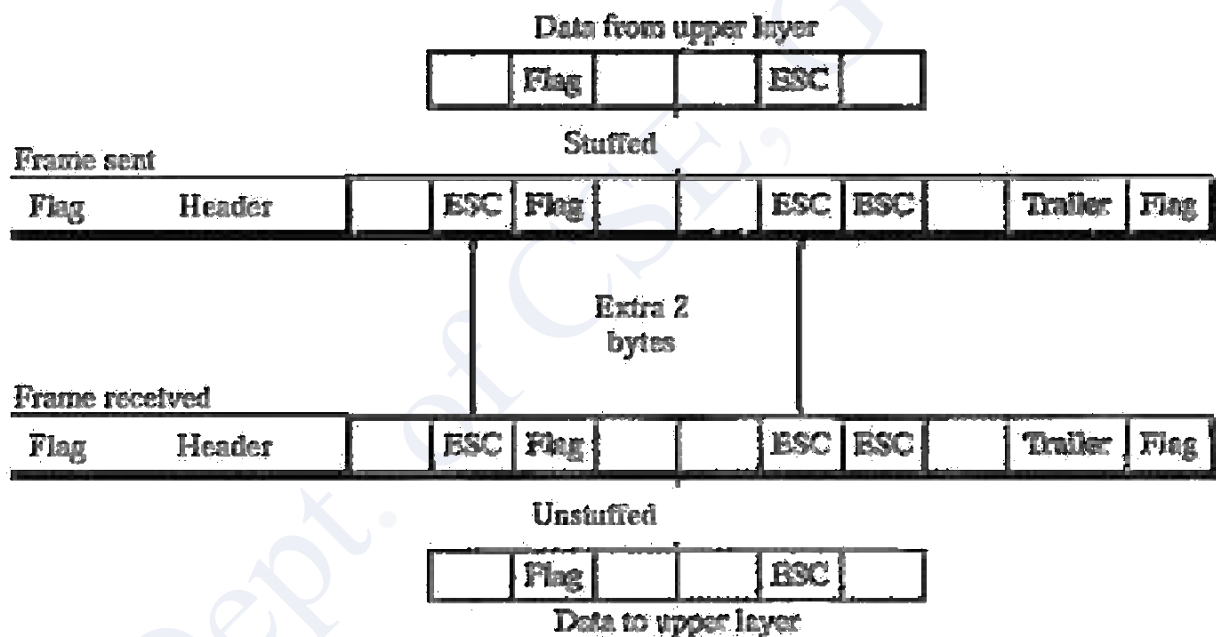
Character-Oriented Protocols

In a character-oriented protocol, data to be carried are 8-bit characters.

The header, which normally carries the source and destination addresses and other control information, and the trailer, which carries error detection or error correction redundant bits, are also multiples of 8 bits.

To separate one frame from the next, an 8-bit (1-byte) flag is added at the beginning and the end of a frame. The flag, composed of protocol-dependent special characters, signals the start or end of a frame.

Any pattern used for the flag could also be part of the information. If this happens, the receiver, when it encounters this pattern in the middle of the data, thinks it has reached the end of the frame. To fix this problem, a **byte-stuffing** strategy was added to character-oriented framing. In byte stuffing (or character stuffing), a special byte is added to the data section of the frame when there is a character with the same pattern as the flag. The data section is stuffed with an extra byte. This byte is usually called the escape character (ESC), which has a predefined bit pattern. Whenever the receiver encounters the ESC character, it removes it from the data section and treats the next character as data, not a delimiting flag.

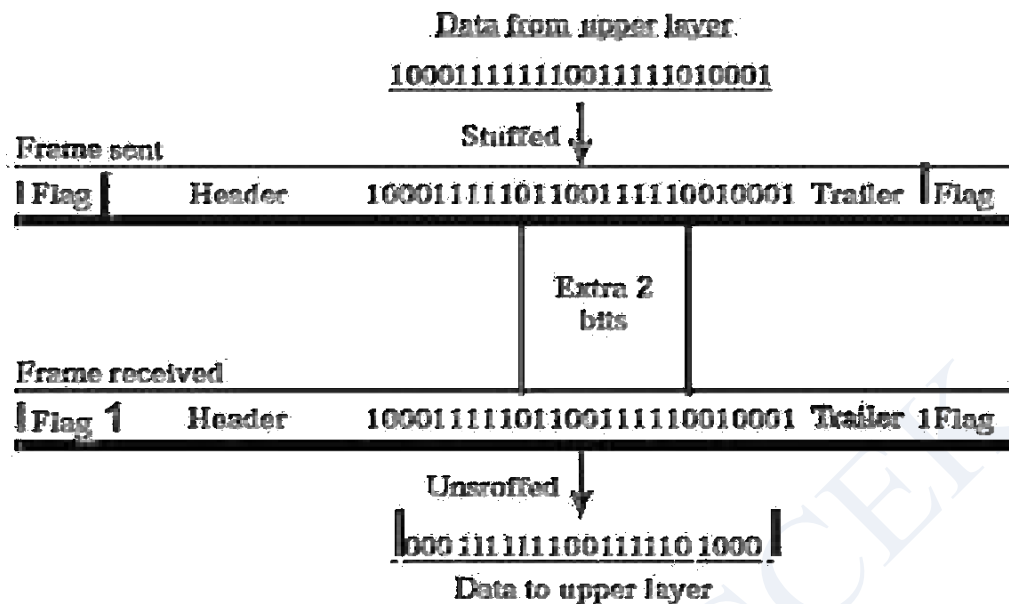


Bit-Oriented Protocols

In a bit-oriented protocol, the data section of a frame is a sequence of bits to be interpreted by the upper layer as text, graphic, audio, video, and so on.

Most protocols use a special 8-bit pattern flag 01111110 as the delimiter to define the beginning and the end of the frame.

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.



Flow and Error Control

Data communication requires at least two devices working together, one to send and the other to receive.

The most important responsibilities of the data link layer are **flow control** and **error control**. Collectively, these functions are known as **data link control**.

Flow Control

Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.

Each receiving device has a block of memory, called a buffer, reserved for storing incoming data until they are processed. If the buffer begins to fill up, the receiver must be able to tell the sender to halt transmission until it is once again able to receive.

Error Control

Error control is both error detection and error correction.

It allows the receiver to inform the sender of any frames lost or damaged in transmission and coordinates the retransmission of those frames by the sender.

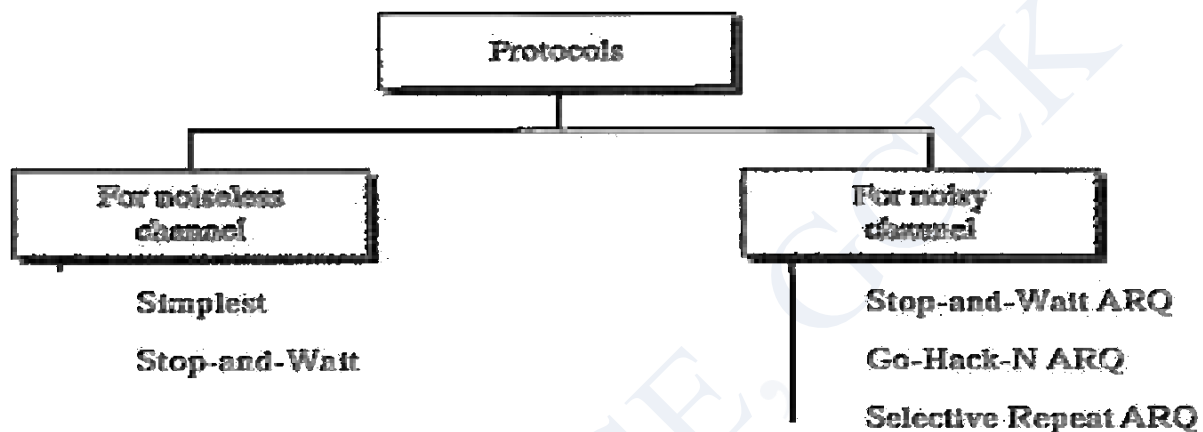
In the data link layer, the term error control refers primarily to methods of error detection and retransmission. This process is called **Automatic Repeat Request (ARQ)**.

Protocols

Data link layer can combine framing, flow control, and error control to achieve the delivery of data from one node to another.

We divide the discussion of protocols into those that can be used for noiseless (error-free) channels and those that can be used for noisy (error-creating) channels.

The protocols in the first category cannot be used in real life, but they serve as a basis for understanding the protocols of noisy channels.



Stop-and- Wait ARQ

To detect and correct corrupted frames, we need to add redundancy bits to our data frame. When the frame arrives at the receiver site, it is checked and if it is corrupted, it is silently discarded. The detection of errors in this protocol is manifested by the silence of the receiver.

When the receiver receives a data frame that is out of order, this means that frames were either lost or duplicated. The corrupted and lost frames need to be resent in this protocol. If the receiver does not respond when there is an error, how can the sender know which frame to resend?

To remedy this problem, the sender keeps a copy of the sent frame. At the same time, it starts a timer. If the timer expires and there is no ACK for the sent frame, the frame is resent, the copy is held, and the timer is restarted.

Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.

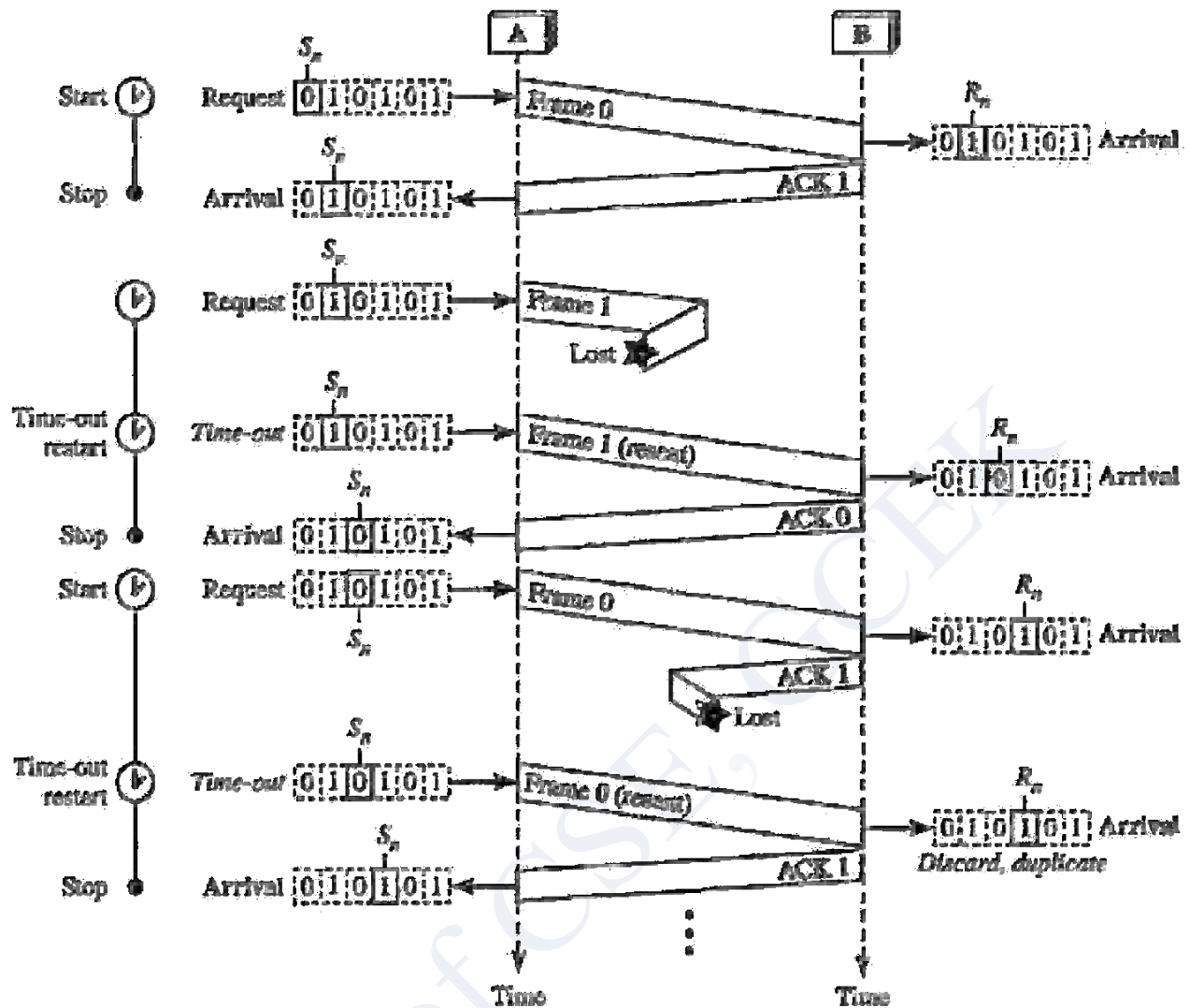
Sequence Numbers

A frame is numbered by sequence numbers. A field is added to the data frame to hold the sequence number of that frame.

If we decide that the field is m bits long, the sequence numbers start from 0, go to $2^m - 1$, and then are repeated.

Let us consider three cases:

1. The frame arrives safe and sound at the receiver site; the receiver sends an acknowledgment. The acknowledgment arrives at the sender site, causing the sender to send the next frame numbered $x + 1$.
2. The frame arrives safe and sound at the receiver site; the receiver sends an acknowledgment, but the acknowledgment is corrupted or lost. The sender resends the frame (numbered x) after the time-out. Note that the frame here is a duplicate. The receiver can recognize this fact because it expects frame $x + 1$ but frame x was received.
3. The frame is corrupted or never arrives at the receiver site; the sender resends the frame (numbered x) after the time-out.



Go-Back-N ARQ

To improve the efficiency of transmission (filling the pipe), multiple frames must be in transition while waiting for acknowledgment. In other words, we need to let more than one frame be outstanding to keep the channel busy while the sender is waiting for acknowledgment.

The first is called Go-Back-N Automatic Repeat Request. In this protocol we can send several frames before receiving acknowledgments; we keep a copy of these frames until the acknowledgments arrive.

In Go-Back-N ARQ, the size of the send window must be less than 2^m ; the size of the receiver window is always 1.

Sequence Numbers

Frames from a sending station are numbered sequentially. However, because we need to include the sequence number of each frame in the header, we need to set a limit. If the header of the frame allows m bits for the sequence number, the sequence numbers range from 0 to $2^m - 1$. For example, if m is 4, the only sequence numbers are 0 through 15 inclusive. However, we can repeat the sequence. So the sequence numbers are

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,

In other words, the sequence numbers are modulo- 2^m . In the Go-Back-N Protocol, the sequence numbers are modulo 2^n , where n is the size of the sequence number field in bits.

Sliding Window

In Go-Back-N Automatic Repeat Request, *the sliding window is an abstract concept that defines the range of sequence numbers that is the concern of the sender and receiver*. In other words, the sender and receiver need to deal with only part of the possible sequence numbers. The range which is the concern of the sender is called the send sliding window; the range that is the concern of the receiver is called the receive sliding window.

The window at any time divides the possible sequence numbers into four regions.

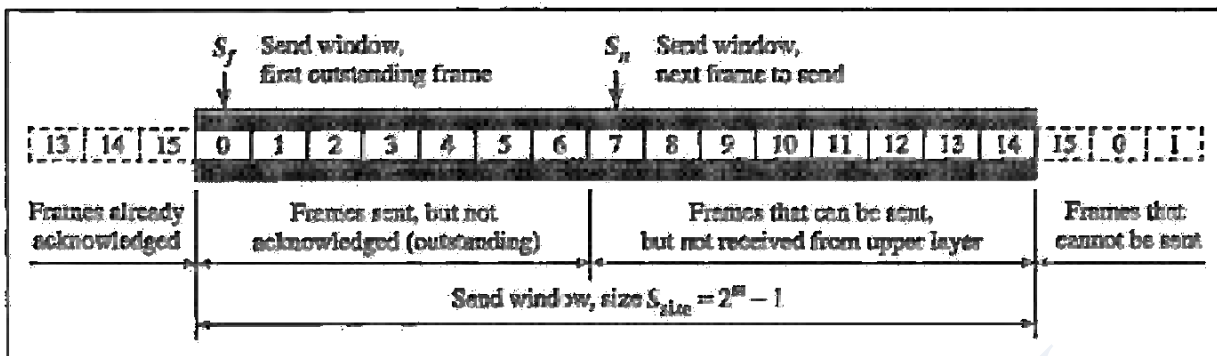
The first region, from the far left to the left wall of the window, defines the sequence numbers belonging to frames that are already acknowledged. The sender does not worry about these frames and keeps no copies of them.

The second region defines the range of sequence numbers belonging to the frames that are sent and have an unknown status. The sender needs to wait to find out if these frames have been received or were lost. We call these outstanding frames.

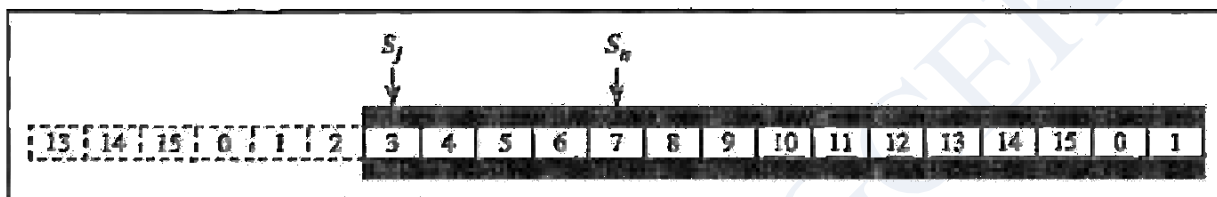
The third range defines the range of sequence numbers for frames that can be sent; however, the corresponding data packets have not yet been received from the network layer.

Finally, the fourth region defines sequence numbers that cannot be used until the window slides.

Send Window for Go Back N ARQ



a. Send window before sliding



b. Send window after sliding

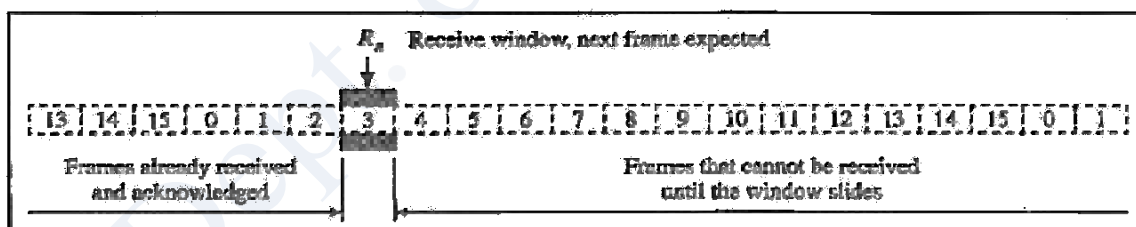
The window itself is an abstraction; three variables define its size and location at any time. We call these variables S_f (send window, the first outstanding frame), S_n (send window, the next frame to be sent), and S_{size} (send window, size).

The variable S_f defines the sequence number of the first (oldest) outstanding frame.

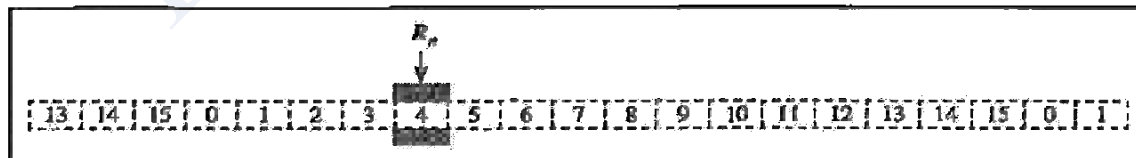
The variable S_n holds the sequence number that will be assigned to the next frame to be sent.

Finally, the variable S_{size} defines the size of the window, which is fixed in our protocol.

Receive Window for Go Back N ARQ



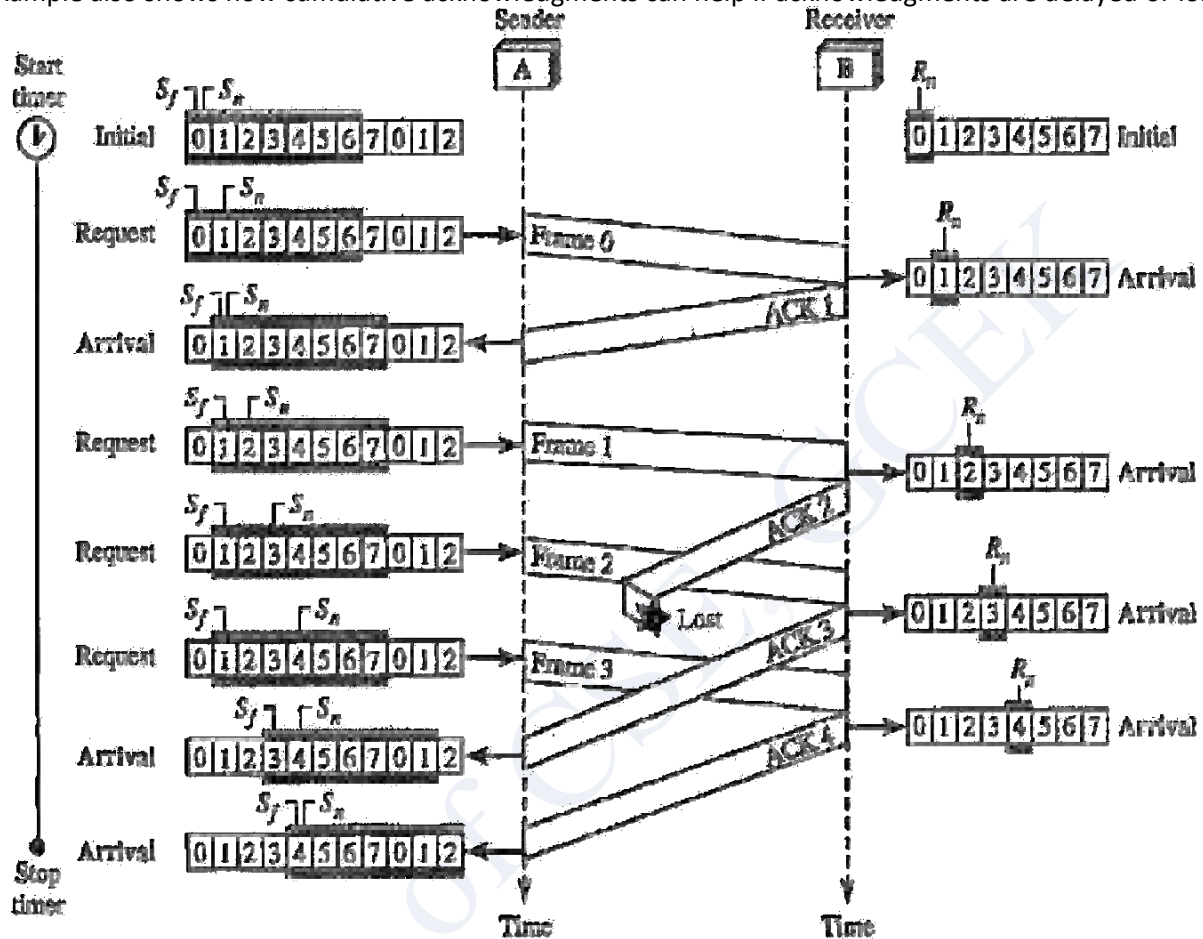
a. Receive window



b. Window after sliding

Example:

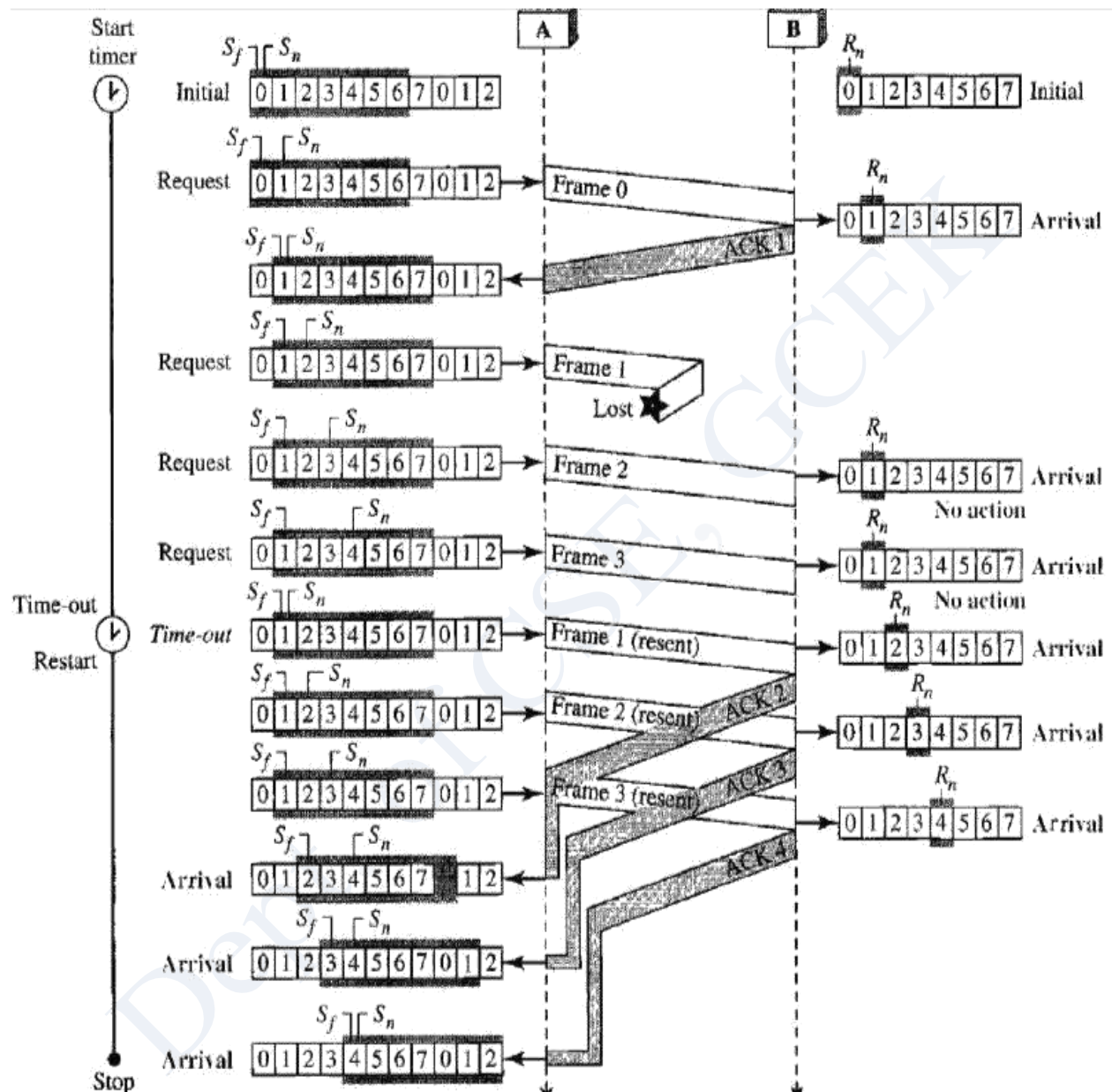
Below figure shows an example of **Go-Back-N**. This is an example of a case where the forward channel is reliable, but the reverse is not. *No data frames are lost, but some ACKs are delayed and one is lost.* The example also shows how cumulative acknowledgments can help if acknowledgments are delayed or lost.



Example:

Below figure shows what happens when a frame is lost. Frames 0, 1, 2, and 3 are sent. However, frame 1 is lost. The receiver receives frames 2 and 3, but they are discarded because they are received out of order (frame 1 is expected). The sender receives no acknowledgment about frames 1, 2, or 3. Its timer finally expires. The sender sends all outstanding frames (1, 2, and 3) because it does not know what is wrong. Note that the resending of frames 1, 2, and 3 is the response to one single event. When the sender is responding to this event, it cannot accept the triggering of other events. This means that when ACK 2 arrives, the sender is still busy with sending frame 3. The physical layer must wait until this event is completed and the data link layer goes back to its sleeping state. We have shown a vertical line to indicate the delay. It is the same story with ACK 3; but when ACK 3 arrives, the sender is busy

responding to ACK 2. It happens again when ACK 4 arrives. Note that before the second timer expires, all outstanding frames have been sent and the timer is stopped.



Go-Back-N ARQ versus Stop-and- Wait ARQ

There is a similarity between Go-Back-NARQ and Stop-and-Wait ARQ. We can say that the Stop-and-WaitARQ Protocol is actually a Go-Back-NARQ in which there are only two sequence numbers and the

send window size is 1. In other words, $m = 1$, $2^m - 1 = 1$. In Go-Back-NARQ, we said that the addition is modulo- 2^m ; in Stop-and-WaitARQ it is 2, which is the same as 2^m when $m = 1$.

Selective Repeat Automatic Repeat Request

Go-Back-N ARQ simplifies the process at the receiver site. The receiver keeps track of only one variable, and there is no need to buffer out-of-order frames; they are simply discarded. However, this protocol is very inefficient for a noisy link. In a noisy link a frame has a higher probability of damage, which means the resending of multiple frames.

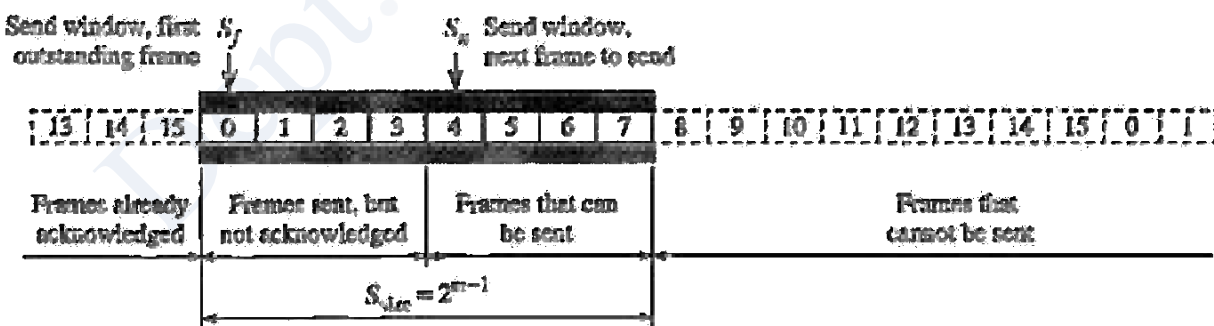
This resending uses up the bandwidth and slows down the transmission. For noisy links, there is another mechanism that does not resend N frames when just one frame is damaged; only the damaged frame is resent. This mechanism is called Selective Repeat ARQ.

Windows

The Selective Repeat Protocol also uses two windows: a send window and a receive window. However, there are differences between the windows in this protocol and the ones in Go-Back-N. First, the size of the send window is much smaller; it is $2^m - 1$. Second, the receive window is the same size as the send window.

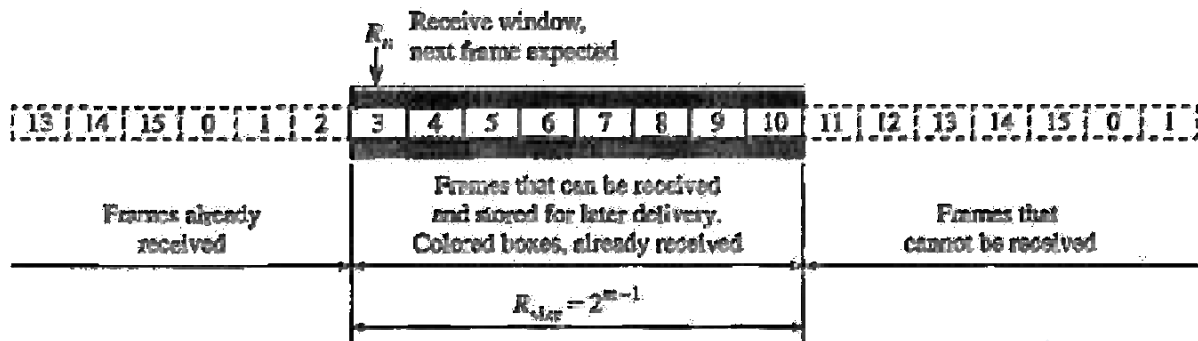
Send window for selective repeat ARQ

The send window maximum size can be $2^m - 1$. For example, if $m = 4$, the sequence numbers go from 0 to 15, but the size of the window is just 8 (it is 15 in the Go-Back-N Protocol). The smaller window size means less efficiency in filling the pipe, but the fact that there are fewer duplicate frames can compensate for this. The protocol uses the same variables as we discussed for Go-Back-N.



Receive window for selective repeat ARQ

The receive window in Selective Repeat is totally different from the one in GoBack-N. First, the size of the receive window is the same as the size of the send window ($2^m - 1$).

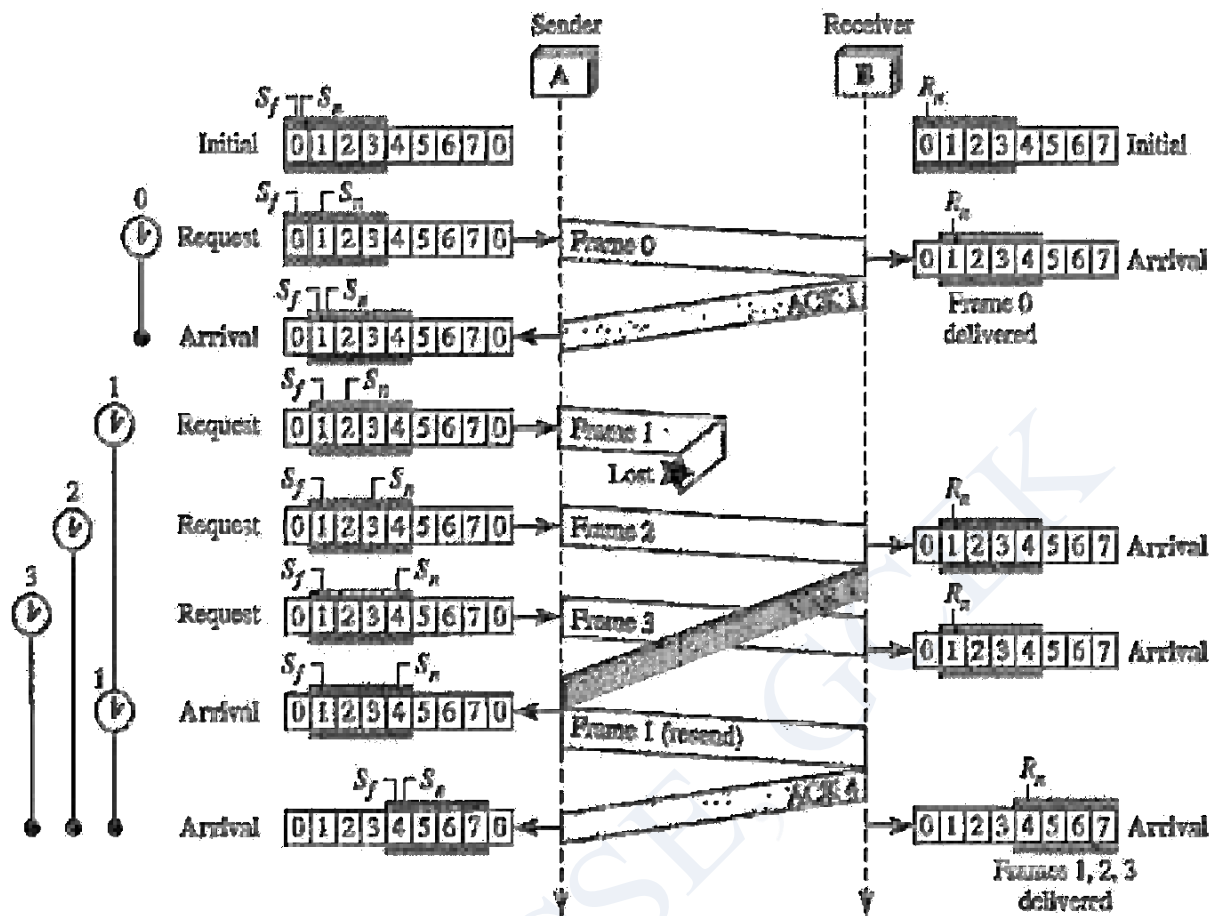


Piggybacking

Data frames flow in only one direction although control information such as ACK and NAK frames can travel in the other direction. In real life, data frames are normally flowing in both directions: from node A to node B and from node B to node A. This means that the control information also needs to flow in both directions.

When a frame is carrying data from A to B, it can also carry control information about arrived (or lost) frames from B; when a frame is carrying data from B to A, it can also carry control information about the arrived (or lost) frames from A. This is called Piggybacking.

Piggybacking is used to improve the efficiency of the bidirectional protocols.



Networks can be categories in to two ways

a) Point to point b) Broad cast channel

- In broadcast network, the key issue is how to share the channel among several users.

- Ex *a conference call with five people*

-Broadcast channels are also called as multi-access channels or random access channels.

-Multi-access channel belong to a sublayer at the DL layer called the MAC sublayer.

The Channel Allocation problem:

a) **Static channel allocation** in LANs & MANs

i) **FDM** ii) **TDM**

Drawbacks: -1) Channel is wasted if one or more stations do not send data.

2) If users increases this will not support.

b) Dynamic channel allocation

i) Pure **ALOHA** & Slotted **ALOHA**

ii) **CSMA**

CSMA/CD

CSMA/CA

Pure ALOHA

-1970's Norman Abramson and his colleagues devised this method, used ground –based radio broad costing. This is called the **ALOHA** system. -The basic idea, many users are competing for the use of a single shared channel.

-There are two versions of ALOHA: **Pure and Slotted.**

-Pure ALOHA does not require global time synchronization, where as in slotted ALOHA the time is divided into discrete slots into which all frames must fit.

-Let users transmit whenever they have data to be sent.

-There will be collisions and all collided frames will be damaged.

-Senders will know through feedback property whether the frame is destroyed or not by listening channel.

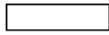
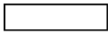
[-With a LAN it is immediate, with a satellite, it will take 270m sec.]

-If the frame was destroyed, the sender waits random amount of time and again sends the frame.

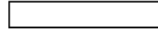
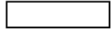
-The waiting time must be random otherwise the same frame will collide over and over.

USER

A



B



C



D



TIME

Frames are transmitted at completely arbitrary times

-Whenever two frames try to occupy the channel at the same time, there will be a collision and both will be destroyed.

-We have to find out what is the efficiency of an ALOHA channel?

-Let us consider an infinite collection of interactive users sitting at their systems (stations).

-A user will always be in two states **typing or waiting**.

-Let the 'Frame time' denote the time required to transmit one fixed length frame.

-Assume that infinite populations of users are generating new frames according to poisson distribution with mean N frames per frame time.

-If $N > 1$ users are generating frames at a higher rate than the channel can handle.

-For reasonable throughput $0 < N < 1$.

-In addition to new frames, the station also generates retransmission of frames.

-Old and new frames are G per frame time.

- $G > N$

-At low load there will be few collisions, so $G \sim N$

-Under all loads, the throughput $S = GP_0$, where P_0 is the probability that a frame does not suffer a collision.

-A frame will not suffer a collision if no other frames are sent with one frame time of its start.

-Let 't' be the time required to send a frame.

-If any other user has generated a frame between time t_0 and t_0+t , the end of that frame will collide with the beginning of the shaded frame.

-Similarly, any other frame started b/w t_0+t and t_0+2t will bump into the end of the shaded frame.

-The probability that 'k' frames are generated during a given frame time is given by the poisson distribution:

$$\Pr[k] = \frac{G^k e^{-G}}{k!}$$

k!

-The probability of zero frames is just e^{-G}

-In an interval two frame times long, the mean number of frames generated is $2G$.

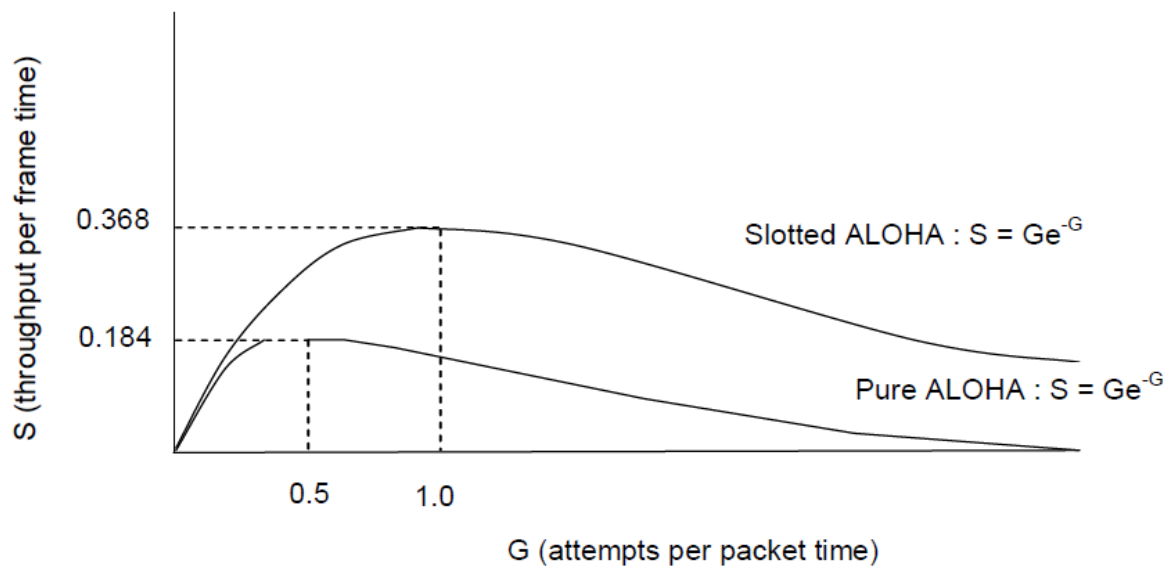
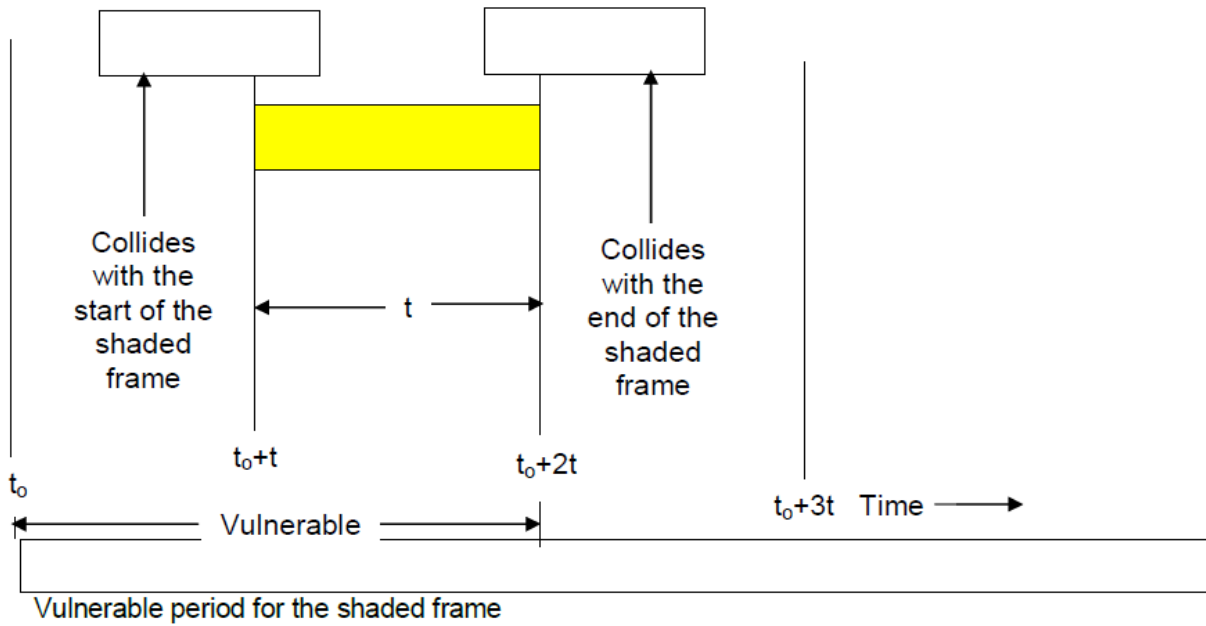
-The probability of no other traffic being initiated during the entire vulnerable period is given by

$$P_0 = e^{-2G}$$

$$S = G e^{-2G} \quad [S = G P_0]$$

The Maximum throughput occurs at $G=0.5$ with $S=1/2e = 0.184$

The channel utilization at pure ALOHA =18%.



Throughput versus offered traffic for ALOHA systems

Slotted ALOHA

- In 1972, Roberts' devised a method for doubling the capacity of ALOHA system.
- In this system the time is divided into discrete intervals, each interval corresponding to one frame.
- One way to achieve synchronization would be to have one special station emit a pip at the start of each interval, like a clock.
- In Roberts' method, which has come to be known as slotted ALOHA, in contrast to Abramson's pure ALOHA; a computer is not permitted to send whenever a carriage return is typed.
- Instead, it is required to wait for the beginning of the next slot.
- Thus the continuous pure ALOHA is turned into a discrete one.
- Since the vulnerable period is now halved, the of no other traffic during the same slot as our test frame is e^{-G} which leads to $S = G e^{-G}$
- At $G=1$, slotted ALOHA will have maximum throughput.
- So $S=1/e$ or about 0.368, twice that of pure ALOHA.
- **The channel utilization is 37% in slotted ALOHA.**

Carrier Sense Multiple Access Protocols

Protocols in which stations listen for a carrier (transmission) and act accordingly are called carrier sense protocols.

Persistent CSMA

When a station has data to send, it first listens to the channel to see if any one else is transmitting at that moment. If the channel is busy, the station waits until it become idle. When the station detects an idle channel, it transmits a frame. If a collision occurs, the station waits a random amount of time and starts all over again. The protocol is called 1-persistent also because the station transmits with a probability of 1 when it finds the channel idle. The propagation delay has an important effect on the performance of the protocol. The longer the propagation delay the worse the performance of the protocol. Even if the propagation delay is zero, there will be collisions. If two stations listen the channel, that is idle at the same, both will send frame and there will be collision.

Non persistent CSMA

In this, before sending, a station sense the channel. If no one else is sending, the station begins doing so it self. However, if the channel is busy, the station does not continually sense it but it waits a random amount of time and repeats the process. This algorithm leads to better channel utilization but longer delays than 1-persistent CSMA. With persistent CSMA, what happens if two stations become active when a third station is busy? Both wait for the active station to finish, then simultaneously launch a packet, resulting a collision. There are two ways to handle this problem.

a) P-persistent CSMA b) exponential backoff.

P-persistent CSMA

The first technique is for a waiting station not to launch a packet immediately when the channel becomes idle, but first toss a coin, and send a packet only if the coin comes up heads. If the coin comes up tails, the station waits for some time (one slot for slotted CSMA), then repeats the process. The idea is that if two stations are both waiting for the medium, this reduces the chance of a collision from 100% to 25%. A simple generalization of the scheme is to use a biased coin, so that the probability of sending a packet when the medium becomes idle is not 0.5, but p , where $0 < p < 1$. We call such a scheme **P-persistent CSMA**. The original scheme, where $p=1$, is thus called 1-persistent CSMA.

Exponential backoff

The key idea is that each station, after transmitting a packet, checks whether the packet transmission was successful. Successful transmission is indicated either by an explicit acknowledgement from the receiver or the absence of a signal from a collision detection circuit. If the transmission is successful, the station is done. Otherwise, the station retransmits the packet, simultaneously realizing that at least one other station is also contending for the medium. To prevent its retransmission from colliding with the other station's retransmission, each station backs off (that is, idles) for a random time chosen from the interval $[0, 2 \cdot \text{max_propagation_delay}]$ before retransmitting its packet. If the retransmission also fails, then the station backs off for a random time in the interval $[0, 4 \cdot \text{max_propagation_delay}]$, and tries again. Each subsequent collision doubles the backoff interval length, until the retransmission finally succeeds. On a successful transmission, the backoff interval is reset to the initial value. We call this type of backoff exponential backoff.

CSMA/CA

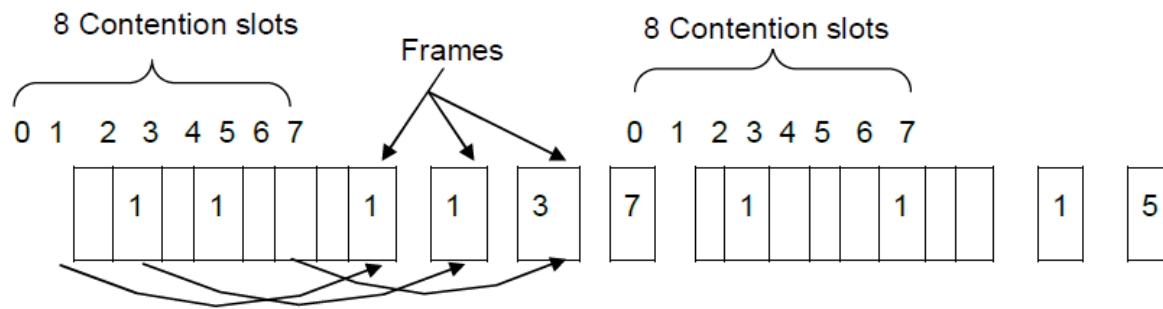
In many wireless LANS, unlike wired LANS, the station has no idea whether the packet collided with another packet or not until it receives an acknowledgement from receiver. In this situation, collisions have a greater effect on performance than with CSMA/CD, where colliding packets can be quickly detected and aborted. Thus, it makes sense to try to avoid collisions, if possible. CSMA/CA is basically p-persistence, with the twist that when the medium becomes idle, a station must wait for a time called the interframe spacing or IFS before contending for a slot. A station gets a higher priority if it is allocated smaller inter frame spacing. When a station wants to transmit data, it first checks if the medium is busy. If it is, it continuously senses the medium, waiting for it to become idle. When the medium becomes idle, the station first waits for an interframe spacing corresponding to its priority level, then sets a contention timer to a time interval randomly selected in the range

$[0, CW]$, where CW is a predefined contention window length. When this timer expires, it transmits a packet and waits for the receiver to send an ack. If no ack is received, the packet is assumed lost to collision, and the source tries again, choosing a contention timer at random from an interval twice as long as the one before (binary exponential backoff). If the station senses that another station has begun transmission while it was waiting for the expiration of the contention timer, it does not reset its timer, but merely freezes it, and restarts the countdown when the packet completes transmission. In this way, stations that happen to choose a longer timer value get higher priority in the next round of contention.

Collision-Free Protocols

A Bit-Map Protocol

In the basic bit-map method, each contention period consists of exactly N slots. If station 0 has a frame to send, it transmits a 1 bit during the zeroth slot. No other station is allowed to transmit during this slot. Regardless of what station 0 does, station 1 gets the opportunity to transmit a 1 during slot 1, but only if it has a frame queued. In general, station j may announce the fact that it has a frame to send by inserting a 1 bit into slot j . After all N slots have passed by, each station has complete knowledge of which stations wish to transmit.

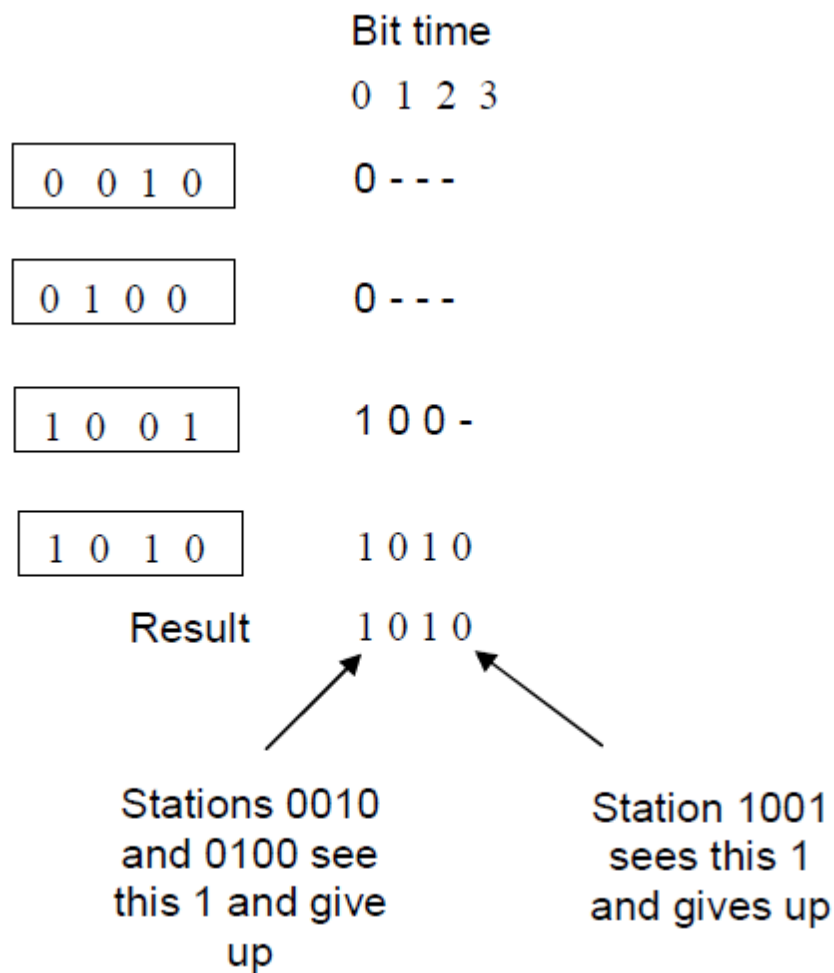


The basic bit-map protocol

Since everyone agrees on who goes next, there will never be any collisions. After the last ready station has transmitted its frame, an event all stations can easily monitor, another N bit contention period is begun. If a station becomes ready just after its bit slot has passed by, it is out of luck and must remain silent until every station has had a chance and the bit map has come around again. Protocols like this in which the desire to transmit is broadcast before the actual transmission are called reservation protocols.

Binary Countdown

A problem with the basic bit-map protocol is that the overhead is 1 bit per station. A station wanting to use the channel now broadcasts its address as a binary bit string, starting with the high-order bit. All addresses are assumed to be the same length. The bits in each address position from different stations are BOOLEAN ORed together. We will call this protocol binary countdown. It is used in Datakit. As soon as a station sees that a high-order bit position that is 0 in its address has been overwritten with a 1, it gives up. For example, if stations 0010, 0100, 1001, and 1010 are all trying to get the channel, in the first bit time the stations transmit 0, 0, 1, and 1, respectively. Stations 0010 and 0100 see the 1 and know that a higher-numbered station is competing for the channel, so they give up for the current round. Stations 1001 and 1010 continue. The next bit is 0, and both stations continue. The next bit is 1, so station 1001 gives up. The winner is station 1010, because it has the highest address. After winning the bidding, it may now transmit a frame, after which another bidding cycle starts. **The binary countdown protocol. A dash indicates silence**



The IEEE 802.3 is for a 1-persistent CSMA/CD LAN. Xerox built a 2.94 Mbps CSMA/CD system to connect over 100 personal workstations on 1-Km cable. This system was called Ethernet through which electromagnetic radiation was once thought to propagate. Xerox DEC and Intel came with another standard for 100 Mbps Ethernet. This differs from old one that it runs at speeds from 1 to 10 Mbps on various media. The second difference between these two is in one header.